Async / Await and Async Rust

An alternative to stackful coroutines

Who am I?

Vic Shihang Li

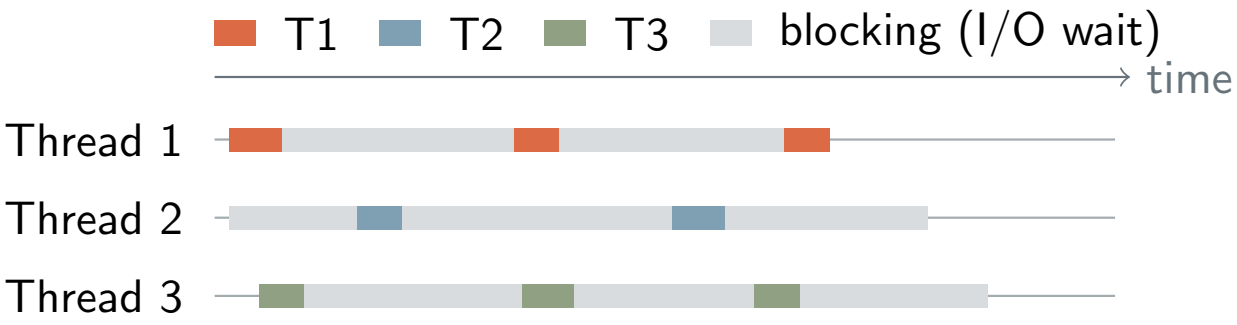
3rd year PhD student studying systems and networking.

Interested in studying how to manage asynchrony in systems
(in the OS, networked services, LLM serving, LLM agents, etc.).

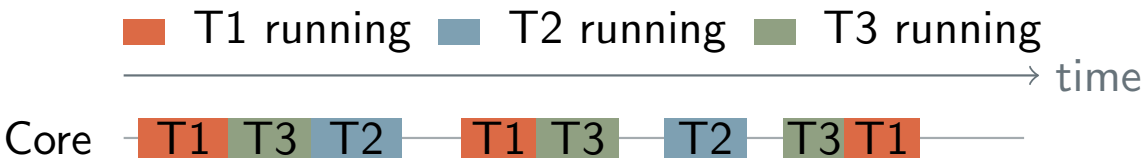
(Talk to me afterwards if you want to learn more about my research!)

Recap: why coroutines (a.k.a. user-level threads)

Many tasks have short CPU bursts + long I/O (network, disk, ...)

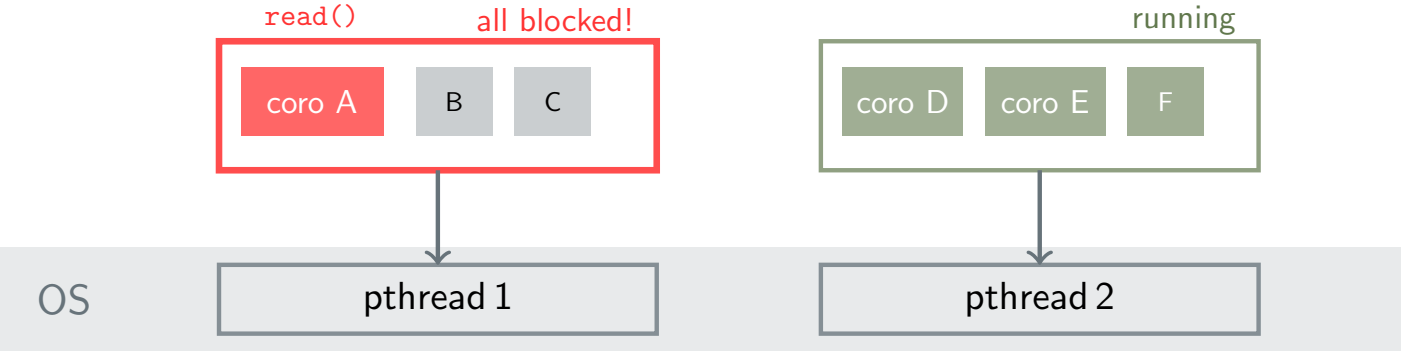


Physical view (single CPU core)



What does "blocking" mean for a coroutine runtime?

A **blocking syscall** from one coroutine blocks the **entire pthread** — blocking every coroutine on it.



To let other coroutines make progress, a blocked coroutine must **yield** instead of blocking, telling the runtime "I am blocked."

What must be saved when a coroutine yields?

To resume later, the runtime must save:

- **Registers**: program counter, stack pointer, general-purpose registers
- **The call stack**: local variables, return addresses, spilled registers

The stack is expensive in terms of memory usage — it stays allocated the *entire* time the coroutine is suspended.

The stack size dilemma

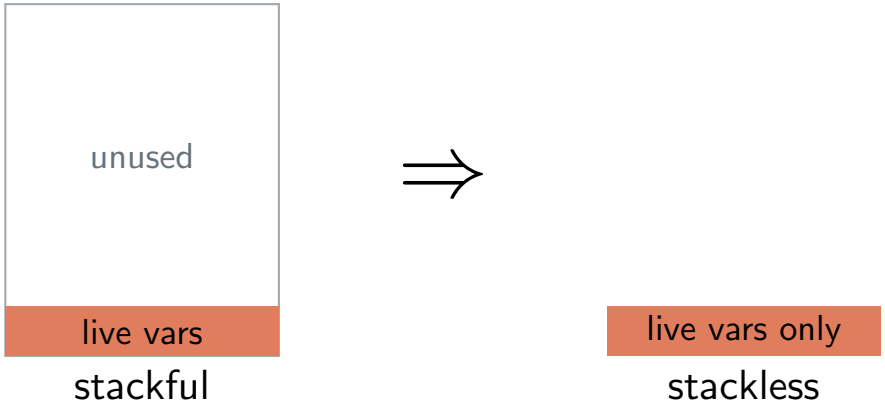
Millions of concurrent connections ⇒ millions of stacks.

Approach	**Stack size**	**1 M conn.**	**Cost**
Linux `pthread`	8 MB	**8 TB**	Enormous waste
Go goroutine	2–8 KB	2–8 GB	Unpredictable resize cost

- **Pre-allocate large** (Linux) — predictable, but most memory *unused*.
- **Start small, grow** (Go) — less waste, but resizing adds *unpredictable latency*.

What does a suspended coroutine actually need?

When a coroutine suspends, most of its stack is wasted:



Key insight: save *only* the live variables across yield points — a **perfectly sized "stack"** with no wasted space.

Overview

1. Designing a stackless coroutine
2. Rust Async / Await Detail

Designing a stackless coroutine

Example async pseudocode

```
async fn api_handler() -> String {
  let course = get_course().await;
  let rating = get_course_rating(course).await;
  format!("class {}: {}/{}", course.name, rating.curr, rating.max
    )
}

async fn get_course() -> Course {
  // db access...
  Course { name: String, desc: String }
}

async fn get_course_rating(course: Course) -> Rating {
  // db access...
  Rating { curr: 5, max: 5 }
}
```

State needed across transition points

Example async pseudocode

```
async fn api_handler() -> String {
  // State saved during first blocking call?
  let course = get_course().await;

  // State saved during second blocking call?
  let rating = get_course_rating(course).await;

  // State after the second blocking call?
  format!("class {}: {}/{}", course.name, rating.curr, rating.max
    )
}
```

State needed across transition points

Example async pseudocode

```
async fn api_handler() -> String {
  // Start { }
  let course = get_course().await;

  // Rating { course }
  let rating = get_course_rating(course).await;

  // Done { course, rating }
  format!("class {}: {}/{}", course.name, rating.curr, rating.max
    )
}
```

State machine (attempt 1)

Each `.await` is a **transition edge**. Each state stores only the **live variables** at that point.

Hand-written stackless coroutine

Example async pseudocode

```
enum ApiHandlerState {
  Start,
  Rating { course: Course },
  Done { output: String },
}

fn api_handler() -> String {
  /* What to do when invoked functions block? */
  self.state = match self.state {
    ApiHandlerState::Start => {
      let course = /* invoke get_course; may block */
      ApiHandlerState::Rating { course }
    }
    ApiHandlerState::Rating { course } => {
      let rating = /* invoke get_course_rating; may block */
      let output = format!("class {}: {}/{}", ...);
      ApiHandlerState::Done { output }
    }
    ApiHandlerState::Done { output } => return output,
  };
}
```

A unified API for suspending / resuming a coroutine

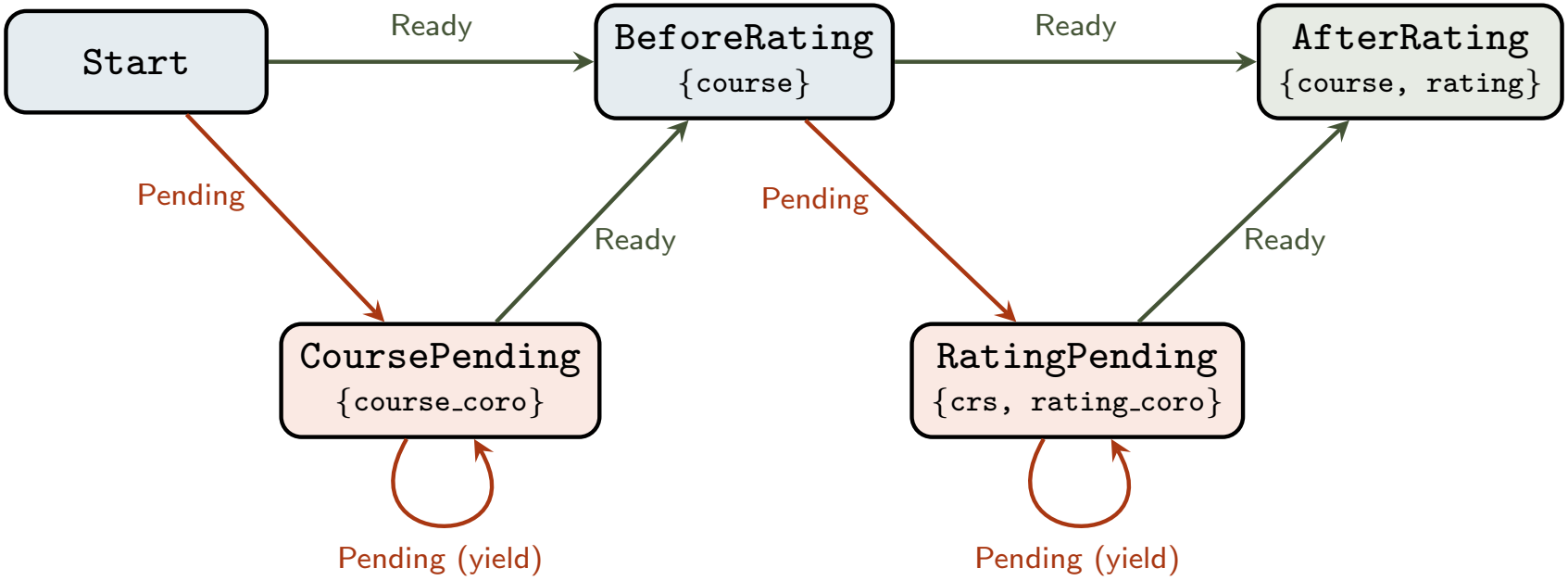
Example async pseudocode

```
enum CoroStatus<T> {
  Ready(T),  // the coroutine is done; here is the final result
  Pending,   // not done yet; poll again later
}

trait Coroutine {
  type Output;
  fn run(&mut self) -> CoroStatus<Self::Output>;
}
```

- `Ready(T)` means the coroutine has **completed** and is returning its final value — not that it is "ready to run again."

Full state machine with pending states



When a nested coroutine returns `Pending`, store it and yield.
On the next poll, resume from the pending state.

The compiler does this for you

We just built a state machine by hand. The Rust compiler does it automatically:

1. Identify live state across each `.await`
2. Build an enum with one variant per suspend point
3. Generate a method like `Coroutine::run` that advances the state machine

A simple scheduler

Example scheduler running stackless coroutines

```
/* Why is this coroutine design "stackless"? */
/* Whose stack is this code running on? */
let run_queue = VecDeque::new();
while let Some(coro) = run_queue.pop() {
  let status = coro.run();
  match status {
    Ready(_) => { /* Coro done running; drop it. */ },
    Pending => run_queue.push(coro),
  }
}
```

Why is this design "stackless"?

- **Stackful**: each coroutine has its own *call stack*. Suspension saves registers + stack pointer; the entire stack stays allocated while the coroutine is suspended.
- **Stackless**: each coroutine is a *state machine struct* that stores only the variables live across `.await` points — no reserved stack.
- Nested calls (`get_course`, `get_course_rating`) are *fields* inside the parent state machine, not frames on a separate stack.

Problem: what happens when a coroutine isn't ready?

Our simple scheduler

```
let run_queue = VecDeque::new();
while let Some(coro) = run_queue.pop() {
  let status = coro.run();
  match status {
    Ready(_) => { /* Coro done running; drop it. */ },
    Pending => run_queue.push(coro),
  }
}
```

What is wrong with this scheduler?

(Hint: what happens when every coroutine in the queue returns `Pending`?)

Waker: avoiding busy-polling

Use Waker to arrange resumption in the future

```
trait Coroutine {
  type Output;
  fn run(&mut self, waker: Waker) -> CoroStatus<Self::Output>;
}
trait Waker {
  // register for wakeup (e.g., lmk when read() has new data)
  fn register(&mut self) { /* ... */ }
  // invoked by the I/O layer once the operation completes
  fn wake(&mut self) { /* re-enqueue the task */ }
}
```

- Who calls `wake()`? The I/O layer (e.g. `epoll`, `io_uring`) calls `wake()` when the operation completes, re-queuing the task for polling.

From our hypothetical coroutine API to Rust's Future

Hypothetical API

```
enum CoroStatus<T> { Ready(T), Pending }
trait Coroutine {
  type Output;
  fn run(&mut self, waker: Waker) -> CoroStatus<Self::Output>;
}
trait Waker { fn wake(&self); }
```

Rust Future (real API)

```
enum Poll<T> { Ready(T), Pending }
trait Future {
  type Output;                /* Waker is in Context */
  fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)
    -> Poll<Self::Output>;
}
struct Waker { /* ... */ }  // concrete struct, not a trait
```

Stackful vs Stackless Coroutines

Property	**Stackful**	**Stackless (async/await)**
Transparent suspend?	Yes (any call can suspend)	No (only at explicit `.await`)
Context switching cost	Higher (save/restore stack)	Lower (poll state machine)
Memory (stacks)	O(coroutines) (one stack per coroutine)	O(cores) (one stack per executor thread)

Overview

1. Designing a stackless coroutine
2. Rust Async / Await Detail

Rust async

Rust gives you the **building blocks** for async, but not an async runtime.

- Language/compiler: `async fn` ⇒ state machine impl `Future`
- Std library: `Future`, `Poll`, `Context`, `Waker` (the contract)
- Missing piece: **a runtime to drive polling and integrate with I/O**

The standard library does *not* choose a scheduler or I/O integration.
Async runtimes are implemented as libraries (e.g., Tokio, async-std, smol).

Hello, World with Tokio

Here's a minimal example of an async main function using Tokio:

```rust
#[tokio::main]
async fn main() {
    println!("Hello from Tokio!");
    // You can .await other async code here.
}
```

The `tokio::main` attribute transforms the entry point so it can `await` async operations.

Futures are lazy: nothing happens until polled

A `Future` is an inert value. Creating it does not start work.

Key idea

No `poll` / `await` ⇒ **no progress**

Common gotcha

```
// async fn called, returns future
let fut = get_course();
// No computation happens here (caveat next slide)
fut.await; // computation happens here
```

Eager work outside the async block

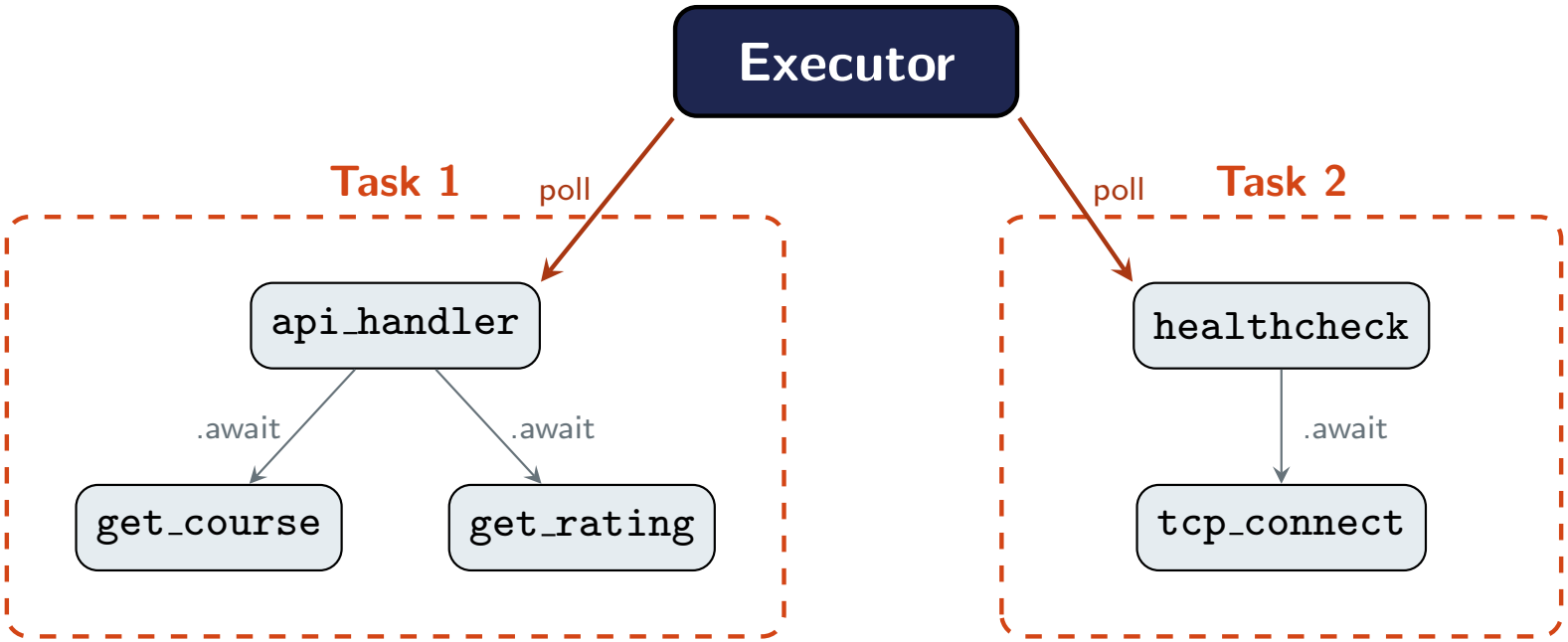
Common pattern in async libraries

```rust
fn make_future() -> impl Future<Output = usize> {
    let start = Instant::now(); // This runs immediately
    println!("Creating future at {start:?}");

    // The async block itself is the future; its body only runs when polled.
    async move {
        println!("Inside future, now being polled");
        // Do some async work here...
        42
    }
}

#[tokio::main]
async fn main() {
    let fut = make_future(); // pre-poll code in ‘make_future‘ runs here
    let result = fut.await;  // this is the first poll
    println!("Result = {result}");
}
```

Future vs Task



- **Future** (blue): passive state machine; composable via `.await`
- **Task** (dashed): a future the **executor** owns and polls

.await vs spawn

- `.await`: run the future as part of the *current* task
- `spawn`: create a *new task* managed by the executor

Example

```rust
#[tokio::main]
async fn main() {
    // Create a future and await it: runs to completion before continuing.
    do_work("awaited").await;

    // Spawn: runs concurrently on the runtime, main keeps going
    let handle = tokio::spawn(async {
        do_work("spawned").await;
    });

    println!("main can do other things while spawned runs");

    // Optionally wait for the spawned task to finish and get its result
    handle.await.unwrap();
}
```

Composing futures: `join!` and `select!`

`join!`: run multiple futures concurrently, wait for *all* to complete.

`join!`: fetch course and rating concurrently

```
let (course, rating) = tokio::join!(
    get_course(),
    get_rating("CSE 451"),
);
```

`select!`: run multiple futures concurrently, take the *first* to complete.

`select!`: race a request against a timeout

```
tokio::select! {
    course = get_course() => println!("got {}", course.name),
    _ = tokio::time::sleep(Duration::from_secs(5)) => {
        println!("timed out!");
    }
}
```

Concurrency without parallelism

`join!`/`select!` give concurrency *within* a single task — no new task, cannot run in parallel.

	`spawn`	`join!`	`select!`
New task?	Yes	No	No
Multi-core?	Yes	No (single thread)	No (single thread)
Semantics	join optionally	wait for *all*	wait for *first*

Cancellation: drop = cancel

In Rust, dropping a future **cancels** it. No special API needed.

Common pattern: `loop { select! }`

```
loop {
  tokio::select! {
    msg = rx.recv()          => handle(msg),
    _   = shutdown.recv()    => break,
  }
}
// Each iteration: losing branch is dropped (cancelled).
// On shutdown, the loop exits and rx.recv() is cancelled.
```

Every iteration, the losing branch's future is **dropped** — no further polls, resources freed immediately.

Cancellation pitfall: lost work in `select!`

Bug: data read from socket but never processed

```
loop {
  tokio::select! {
    data = socket.read()    => process(data),
    _    = shutdown.recv()  => break,
  }
}
// If read() completes *and* shutdown fires in the same poll,
// select! picks one branch -- the other is dropped.
// Data may have been consumed from the socket but never processed!
```

Problem: `read()` may have pulled bytes from the socket internally. If `select!` drops it, those bytes are lost.

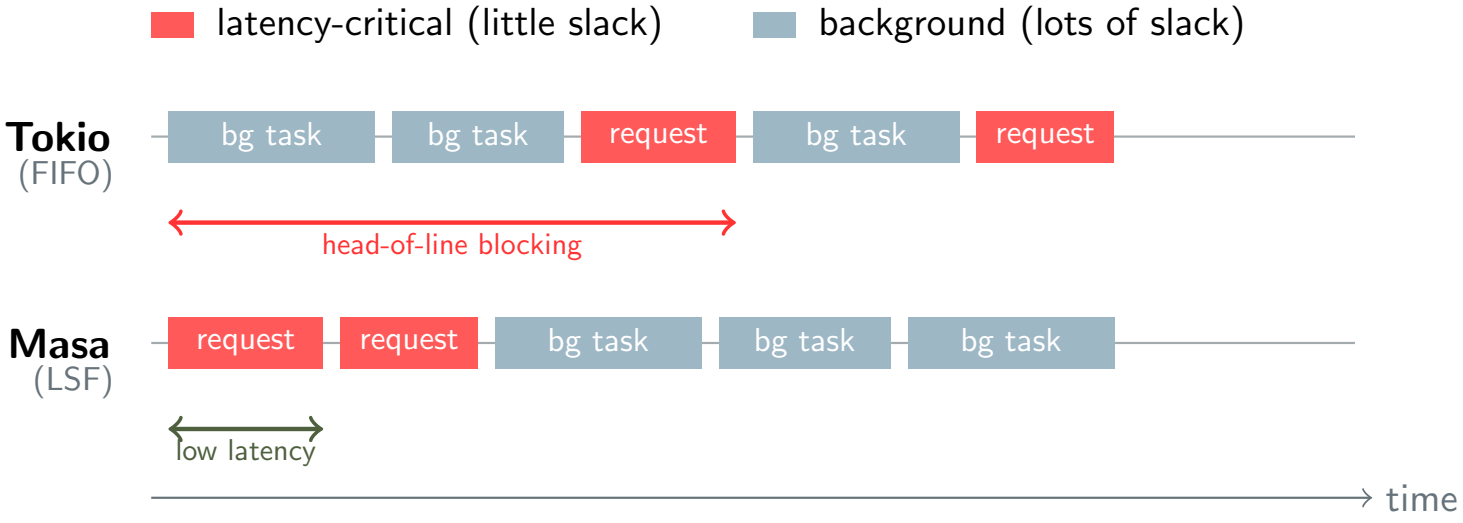
Lesson: a future is **cancellation-safe** if no observable work is lost when it is dropped between polls — i.e. any partial progress it made internally can be safely discarded.

My Research

Applying what we learned to real systems

Masa: slack-aware scheduling for async Rust

Existing user-level threading use FIFO scheduling — low-priority tasks cause **head-of-line blocking** for latency-critical requests.



Masa studies slack-aware scheduling in latency-sensitive web services. Rust's async runtime is not part of the language, making it easy to swap in custom scheduling policies.

Talk to me if you're interested in this research!

Thank You!

You can reach me at
`shli@cs.washington.edu`
vic-li.me

Appendix: Pin

Optional material

What is Pin?

Rust Future

```
enum Poll<T> { Ready(T), Pending }
trait Future {
  type Output;
  fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)
    -> Poll<Self::Output>;
}
```

A slightly more complex async example

Async code with a borrow that must live across a later `.await`

```
async fn api_handler() -> String {
  let course = get_course().await;

  // returns &str into course.desc
  let code = parse_course_code(&course.desc);

  let rating = get_course_rating(code).await;

  // accessing ‘code‘: what’s needed for mem safety?
  format!("class {} rating={}", code, rating)
}
```

Stackless futures can self-reference

Key risk: a compiled async state machine can hold *pointers into itself*.

Building on the state-machine idea

```
enum ApiHandlerState {
  Start,
  Rating {
    // course.desc is a String in the state machine
    course: Course,
    // and we computed a pointer/slice into course.desc
    code: &str,
    rating_fut: RatingFuture,
  },
  Done { output: String },
}
```

If the executor moves this future between polls, `code` becomes dangling.

Example of moving the future

Executor moving the future, causing dangling reference

```
let mut fut = api_handler();  // future value lives on the stack
loop {
  match fut.poll(&mut cx) {
    Ready(out) => break out,
    Pending => {
      // move it into a queue (reallocation / Vec growth / etc.)
      run_queue.push(fut);        // <-- moves fut
      fut = run_queue.pop();      // <-- moves fut again
    }
  }
}
```

Pin: promise that the future will not move

Rust fixes this by requiring polling through a `Pin`:

Real signature (simplified)

```
fn poll(
  self: Pin<&mut Self>,
  cx: &mut Context<'_>
) -> Poll<Self::Output>;
```

- `Pin` says: once placed, this future's address is stable.
- Executors must store pinned futures in a way that won't move them.
- Then internal references (like `code`) remain valid across `.await`.

Question: moving future before first poll

Async executor

```
let mut fut = api_handler();
// Can you move the future before its first poll?
loop {
  match fut.poll(&mut cx) {
    Ready(out) => break out,
    Pending => {
    }
  }
}
```

Answer: yes, moving before first poll is safe

- Self-references are only created *during* a poll (e.g. when code after an `.await` borrows state from before it).
- Before the first poll, the state machine is in its initial variant — no internal pointers exist yet.
- This is why executors can freely move a future into a pinned allocation *before* polling it for the first time:

Heap pinning (common in executors)

```
let fut = api_handler();          // create (movable)
let pinned = Box::pin(fut);       // move into heap, then pin
pinned.as_mut().poll(&mut cx); // first poll: now pinned
```

Stack pinning with `pin!`

You can also pin without heap allocation using the `pin!` macro:

Usage

```
let fut = api_handler();          // create (movable)
let mut fut = pin!(fut);          // pin in place on the stack
fut.as_mut().poll(&mut cx);       // first poll: now pinned
```

How does this work? The macro shadows the original binding:

`pin!` expands roughly to

```
let mut fut = api_handler();   // value on the stack
// shadow ‘fut‘ with a Pin<&mut _> to it
let mut fut = unsafe { Pin::new_unchecked(&mut fut) };
// original ‘fut‘ is no longer accessible by name,
// so safe code cannot move it
```